TP — Intégrité et hachage avec OpenSSL

Sommaire

1. Prérequis . . . 1
2. Partie 1 — Hacher un fichier et vérifier son intégrité . . . 1
3. Exercice 1 — L'effet d'avalanche . . . 2
4. Exercice 2 — La longueur fixe . . . 2
5. Exercice 3 — Corrompu ou non ? . . . 3
6. Exercice 4 — Pourquoi MD5 est-il « cassé » ? . . . 4
7. Exercice 5 — Une collision MD5 en vrai (démonstration) . . . 4
8. Exercice 6 — MD5 est mort, SHA-1 aussi . . . 5
9. Livrable . . . 5

Objectif : appliquer une fonction de hachage à des données pour **vérifier leur intégrité**, et observer les grandes propriétés du hachage (effet d'avalanche, longueur fixe, obsolescence de MD5) directement en ligne de commande.

1. Prérequis

- **Git Bash** (fournit `openssl` et les commandes Unix).
- Le cours **Intégrité et hachage**.



Vérifiez qu'OpenSSL répond : `openssl version`. Toutes les commandes de ce TP se tapent dans **Git Bash**.

2. Partie 1 — Hacher un fichier et vérifier son intégrité

Étape 1. Créez un fichier `messageAuxSIO.txt` (par exemple dans `C:\Temp`) avec le contenu :

```
Les SIO sont drôlement sérieux et forts cette année.
```

Étape 2. Ouvrez **Git Bash**, placez-vous dans le dossier (`cd /c/Temp`) et calculez l'empreinte SHA-256 du fichier :

```
openssl dgst -sha256 messageAuxSIO.txt
```



Figure 1. Hachage d'un fichier avec OpenSSL

Étape 3. Enregistrez l'empreinte dans un fichier pour la transmettre :

```
openssl dgst -sha256 messageAuxSIO.txt > hashSIO.txt
```

Étape 4 — échange en binôme. Donnez à un camarade les **deux** fichiers (`messageAuxSIO.txt` et `hashSIO.txt`). Il recalcule l'empreinte de son côté :

```
openssl dgst -sha256 messageAuxSIO.txt > verif_hashSIO.txt
diff hashSIO.txt verif_hashSIO.txt && echo "INTACT" || echo "ALTERE"
```

Si les empreintes sont **identiques**, le fichier n'a pas été modifié.

3. Exercice 1 — L'effet d'avalanche

Tapez successivement ces deux commandes (seule la casse du « B » change) :

```
echo -n "Bonjour" | openssl dgst -sha256
echo -n "bonjour" | openssl dgst -sha256
```



Figure 2. Un seul caractère modifié → empreinte radicalement différente

Question. Comparez les deux empreintes. Que concluez-vous sur le lien entre une petite modification de l'entrée et la sortie ?

4. Exercice 2 — La longueur fixe

Hachez une entrée **très courte**, puis une entrée **beaucoup plus longue** :

```
echo -n "ok" | openssl dgst -sha256
echo -n "un texte beaucoup plus long que deux caracteres..." | openssl dgst -sha256
```



Figure 3. La sortie a toujours la même longueur (64 caractères hexadécimaux)

Questions.

1. Quelle propriété du hachage cet exercice illustre-t-il ?
2. L'empreinte SHA-256 fait **256 bits**. Combien cela fait-il d'**octets** ? Pourquoi l'affichage compte-t-il alors **64 caractères** et non 32 ?

5. Exercice 3 — Corrompu ou non ?

Simulez la réception d'un fichier, puis son altération d'**un seul accent** :

```
cp messageAuxSIO.txt recu.txt
openssl dgst -sha256 recu.txt          # empreinte à la réception
# ... on retire l'accent de "sérieux" -> "serieux" ...
openssl dgst -sha256 recu.txt          # empreinte après altération
```



Figure 4. La moindre modification change l'empreinte : l'altération est détectée

Question. En quoi ce mécanisme permet-il de **détecter** une altération, même minime ?

6. Exercice 4 — Pourquoi MD5 est-il « cassé » ?

Comparez une empreinte **MD5** et une empreinte **SHA-256** de la même entrée :

```
echo -n "ENC" | openssl dgst -md5
echo -n "ENC" | openssl dgst -sha256
```

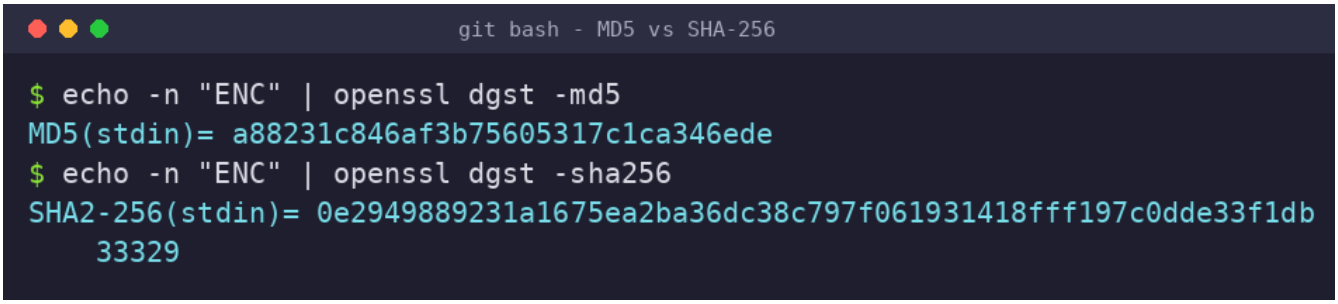


Figure 5. MD5 (128 bits) est plus courte que SHA-256 (256 bits)

Questions.

1. Quelle empreinte est la plus courte ? Pourquoi une empreinte plus courte favorise-t-elle les **collisions** ?
2. Recherchez : pourquoi MD5 est-il aujourd'hui **déconseillé** pour la sécurité ?

7. Exercice 5 — Une collision MD5 en vrai (démonstration)

L'enseignant vous fournit **deux documents PDF différents**, `collision1.pdf` et `collision2.pdf` (source : le dépôt *corkami/collisions* d'Ange Albertini).

Étape 1. Ouvrez les deux fichiers et regardez-les. Ce sont deux articles de recherche sans rapport l'un avec l'autre : titres différents, auteurs différents, nombre de pages différent.

Étape 2. Comparez maintenant leurs empreintes :

```
openssl dgst -md5    collision1.pdf collision2.pdf   # empreintes MD5
openssl dgst -sha256 collision1.pdf collision2.pdf   # empreintes SHA-256
```

Questions.

1. Les deux documents sont-ils identiques ? Qu'est-ce qui les distingue à l'œil ?
2. Leurs empreintes **MD5** sont-elles identiques ? Et leurs empreintes **SHA-256** ?
3. Qu'en concluez-vous sur MD5 ?

4. Une banque vérifie l'intégrité d'un relevé téléchargé en comparant son empreinte MD5 à celle qu'elle a publiée. Que peut faire un attaquant qui a lu cet exercice ?

8. Exercice 6 — MD5 est mort, SHA-1 aussi

L'enseignant vous fournit deux autres PDF, `shattered1.pdf` et `shattered2.pdf` (source : l'attaque *SHAttered*, Google et CWI Amsterdam, 2017).

```
openssl dgst -sha1   shattered1.pdf shattered2.pdf   # empreintes SHA-1
openssl dgst -sha256 shattered1.pdf shattered2.pdf   # empreintes SHA-256
```

Ouvrez les deux documents et comparez la **liste des auteurs** en première page.

Questions.

1. Qu'est-ce qui change entre les deux documents ? Leurs empreintes **SHA-1** changent-elles pour autant ?
2. Ce document est signé électroniquement, et la signature porte sur son empreinte SHA-1. Que vaut cette signature ?
3. Parmi MD5, SHA-1 et SHA-256, lequel utiliseriez-vous aujourd'hui, et pourquoi ?



MD5 est cassé depuis **2005**, SHA-1 depuis **2017**. SHA-256 résiste toujours. Une fonction de hachage n'est pas éternelle : elle vieillit, et il faut en changer avant qu'il ne soit trop tard.

9. Livrable

🗃 À rendre :

- ☐ La preuve de la vérification en binôme (Partie 1 : INTACT / ALTERE).
- ☐ Vos réponses aux exercices 1 à 6.

Table 1. Auto-évaluation

Vérification	OK ?
Je sais hacher un fichier et comparer deux empreintes	[ ]
Je peux expliquer l'effet d'avalanche	[ ]
Je sais pourquoi MD5 ne doit plus être utilisé	[ ]
Je sais ce qu'est une collision, et ce qu'elle permet à un attaquant	[ ]